

Problem 1: Personalized Study Resource Tracker

Problem: Students often use multiple resources (textbooks, online tutorials, video lectures, articles) for a single course or topic, making it difficult to **track progress** and **manage access** links efficiently.

Objective : Build a web application where users (students) can create boards or lists for specific courses/topics.

Features:

Allow users to **add links** to external study materials (YouTube, drive links, specific websites).

Implement a simple **progress tracking** mechanism (e.g., status like 'To Do', 'In Progress', 'Completed', or a simple percentage).

Enable users to **categorize resources** by type (video, reading, practice problem).

Problem 2: Event Scheduling and Volunteer Sign-up App

Problem: Organizing small college/departmental events and coordinating student volunteers often relies on scattered emails and spreadsheets, making the sign-up and management process cumbersome.

Objective: Develop a streamlined web interface for event organizers to post events and for volunteers to sign up for specific shifts/roles.

Features:

Event Creation: A form for the organizer to input event title, date, time, and a brief description.

Volunteer Roles: The organizer should be able to define a fixed number of required volunteer slots/roles (e.g., 'Registration Desk - 3 slots', 'Logistics - 5 slots').

Sign-up Interface: A public-facing view where students can see the remaining available slots and sign up by entering their name/email. The list updates in real-time or upon refresh.

Tech Focus: Basic form handling, simple client-side validation, and a mock/simple backend to handle the sign-up data (e.g., Node.js with a lightweight database like NeDB or just a basic API).

Problem 3: Real-time Dynamic Route Status Dashboard (Logistics & Geo-data)

Problem Statement: A regional logistics company needs a web application to effectively monitor a fleet of delivery vehicles. The current system provides raw GPS data, but lacks a centralized dashboard to visualize **real-time vehicle locations**, highlight **delivery status changes**, and dynamically **identify vehicles experiencing delays** (e.g., stationary for too long, or far off the optimized route).

Objective

To build a single-page monitoring dashboard that simulates real-time data ingestion for multiple vehicles, displays their current status on a map, and uses immediate visual feedback to alert dispatchers to potential issues.

Features

Simulated Real-time Data Feed: Receive updated location and status data for 10-15 vehicles every few seconds.

Interactive Map Visualization: Display moving vehicle markers on an **interactive map**.

Dynamic Status Highlighting: Change marker color/icon to indicate status changes (e.g., **red** for delayed/stationary).

Vehicle Detail Panel: Display detailed information (Driver ID, speed, time since last stop) upon clicking a marker.

Tech Focus

- **Frontend Framework:** React, Vue, or Angular for state management.
- **Real-time Simulation:** Use **WebSockets** (or simulated interval polling).
- **Mapping API:** Integration of a **Maps API** for dynamic geo-data visualization.
- **Logic:** Advanced **JavaScript logic** for implementing delay-detection rules.

Problem 4: Complete OFDM Communication System Simulation (Scilab)

Objective: Design and simulate a complete **Orthogonal Frequency Division Multiplexing (OFDM)** Transmitter–Receiver chain.

Tasks:

1. Implement modulation (QPSK and 16-QAM).
2. Apply IFFT (TX) and FFT (RX).
3. Add and remove cyclic prefix.
4. Add AWGN channel.
5. Plot **BER vs SNR** for both modulations.
6. Compare performance and write conclusion.

Expected Output:

- Graphs, plots, Scilab *.sce* file
- Flowcharts + explanation
- BER performance table

Problem 5 : 32-bit Mini RISC Processor Design (Verilog)

Objective:

Statement: Design and simulate a simplified **32-bit RISC CPU**.

Minimum Requirements:

- Instruction Memory
- Register File
- ALU (ADD, SUB, AND, OR)
- LOAD, STORE, JMP
- Fetch–Decode–Execute cycle

Expected Output:

- Verilog code
- Testbench
- Timing diagrams

Block diagram description

Problem 6: Automatic Number Plate Recognition (ANPR) System Using Scilab

Objective: Develop a software-only ANPR using image processing algorithms.

Tasks:

1. Pre-processing: grayscale, thresholding, noise removal.
2. Edge detection & segmentation.
3. Extract each character.
4. Recognize characters using template matching.
5. Output recognized vehicle number.

Expected Output:

- Stepwise image results
- Detected number
- Code + report